Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

EXPERIMENT – 6

AIM: To study and Implement Agile Software Project lifecycle using Jira with DevOps Pipeline Integration.

LO MAPPED: LO1.

LO1: Understand and apply Agile software development methodologies and project tracking tools to plan, manage and track the software project lifecycle.

THEORY:

Agile Software Development

Agile is an approach to software development in which work is carried out in short repeated cycles instead of one long phase. Requirements are broken into small pieces that can be delivered independently, and working software is produced at the end of every cycle. Because the product is reviewed after each cycle, a change in requirement can be accepted at any stage rather than only at the start of the project.

Agile Software Project Lifecycle

The Agile lifecycle repeats the same phases for every iteration. Requirements are collected and recorded in a backlog, a subset is selected during sprint planning, the selected items are developed and tested, the increment is reviewed, and the work is released. Items not selected remain in the backlog for a later iteration, so the project advances through a series of small completed increments.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Scrum

Scrum is the Agile framework used in this experiment. Work is organised into fixed-length iterations called sprints. The product backlog holds all known requirements, and the items chosen for the current iteration form the sprint backlog. Progress is tracked on a board during the sprint, and the sprint is formally closed when the selected items are complete.

Scrum Board

A Scrum board shows the issues of the active sprint arranged in columns, one column for each state of the workflow. The board used in this experiment has the columns To Do, In Progress, In Review and Done. Moving a card from one column to the next records the transition against the issue, so the board always shows the current state of the sprint.

User Stories and Story Points

A user story describes a requirement from the point of view of the person who will use the software, and is small enough to be finished within a single sprint. A task is technical work that does not describe user-facing behaviour, such as configuring a build server. Story points are a relative estimate of the effort a story needs, and adding the points of the selected stories gives the planned size of the sprint.

Jira

Jira is a project tracking tool used to manage Agile projects. Each requirement is recorded as an issue with a unique issue key, a type, an assignee, an estimate and a status. Issues are shown on the board and moved through the workflow as work proceeds. When a sprint is completed, Jira produces reports from the recorded data.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

DevOps

DevOps is a set of practices that brings development and operations together so that software can be built, tested and released frequently and reliably. It depends on automation, so that steps which would otherwise be carried out by hand are performed by tools in the same order every time.

Continuous Integration and Continuous Deployment

Continuous Integration is the practice of adding code changes to a shared repository regularly, where each change is automatically built and checked so that errors are found early. Continuous Deployment extends this by automatically delivering the code that passes those checks to the target environment.

Git and GitHub

Git is a distributed version control system that records the history of changes to the source code and allows each feature to be developed on its own branch. GitHub is a remote hosting service for Git repositories, and it is the repository from which the automation server obtains the application files and the pipeline definition.

GitHub Actions

GitHub Actions runs automated workflows stored with the application in .github/workflows. A workflow defines its events, jobs, permissions and steps. Hosted runners check out the source, execute the tests and build the site. The workflow used here is site.yml, so changes to the delivery process are versioned with the application.

Event Triggers

A pull_request event runs validation before a change is merged. A push to main runs validation again and then deploys the successful artifact. These events start the workflow automatically without a local automation server or a polling schedule.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

GitHub Pages

GitHub Pages publishes static HTML, CSS and JavaScript. The build job uploads the dist directory as an artifact, and a dependent deployment job publishes that artifact. This experiment uses a static browser application with localStorage; it does not require a server-side database or a Docker container.

Integration of Jira with the DevOps Pipeline

Traceability is established through Jira keys in the feature branch, commits and pull request title. SWD-1 identifies the customer support epic and SWD-2 identifies delivery automation. The repository history and workflow runs can therefore be related to the planned Jira work. This execution uses issue-key traceability and manual Jira status updates; automatic synchronization through the GitHub for Jira app was not configured.

Agile and DevOps Lifecycle Flow:

Requirement recorded in Jira → Work selected for a sprint → Branch and commits identify Jira keys → Pull request triggers tests → Merge to main triggers test and build → Successful artifact deployed to GitHub Pages → Result verified and Jira work marked Done → Sprint completed.

EXECUTION:

The application used in this experiment is Sheify Customer Support, a static web prototype for the group Atharva, Utsav and Mohit. It provides support request validation, browser storage, status filtering, request closure and FAQs. Jira project SWD records the work, GitHub stores the source, and GitHub Actions tests, builds and deploys the application to GitHub Pages.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Step 1: Create a Jira Account and a Scrum Project

A team-managed Scrum project named Sheify Website Development was created in Jira with the key SWD. Two epics group the requirements: SWD-1 Customer Support and SWD-2 Delivery Automation. The initial project screen is shown below.

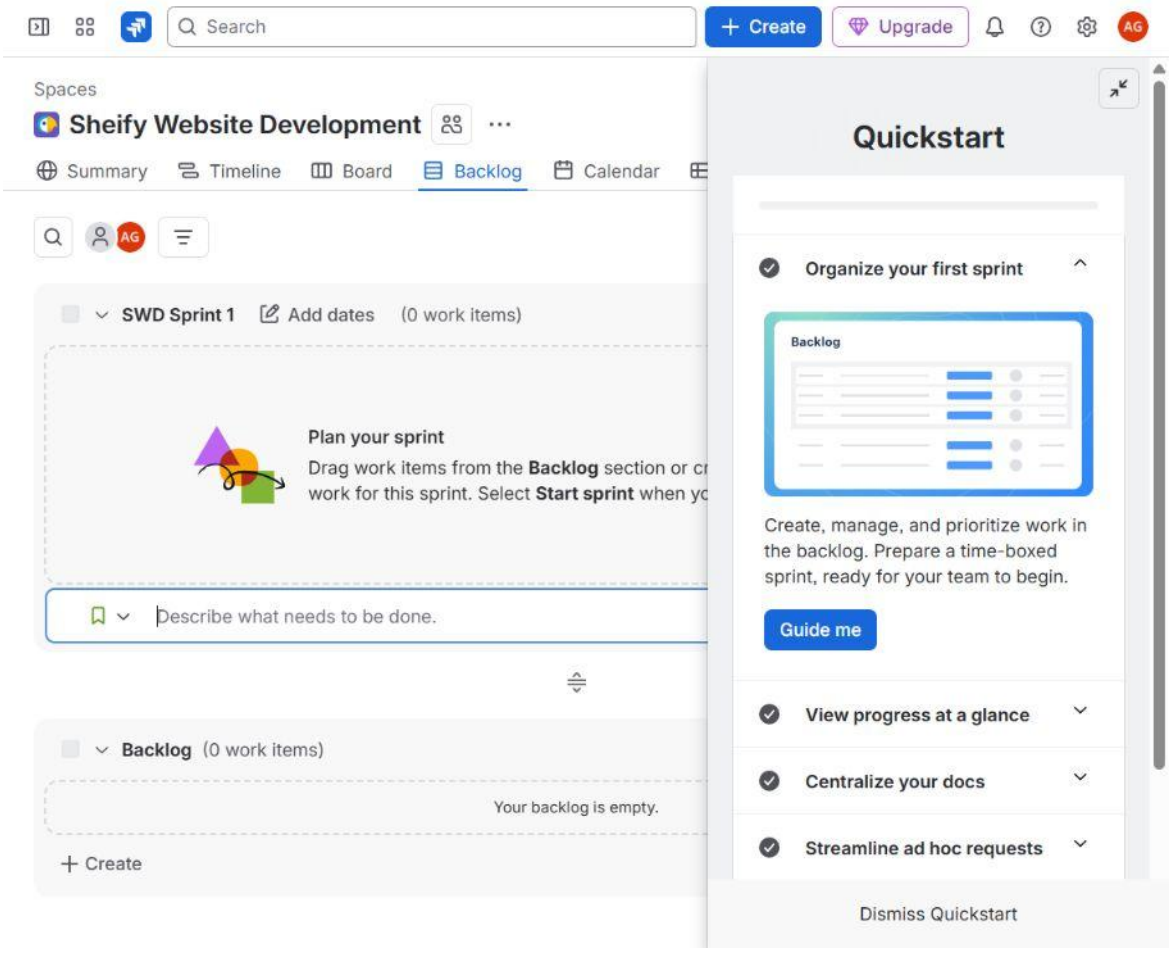


Figure 1: Initial Sheify Website Development project

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Step 2: Understand the Scrum Board

The board contains To Do, In Progress, In Review and Done columns. Starting SWD Sprint 1 placed the six selected work items on the board. Status changes record progress; the board screenshot below shows the initial active sprint.

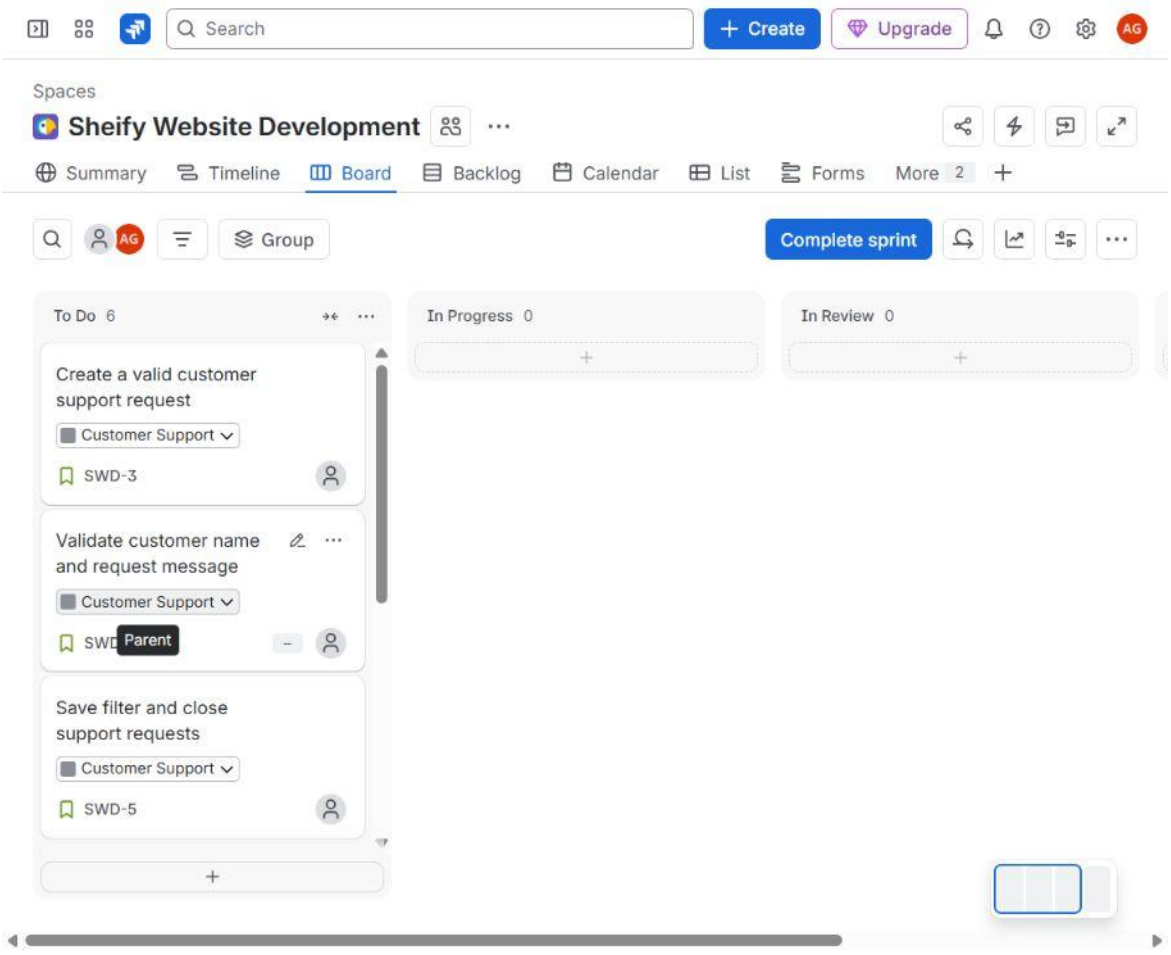


Figure 2: Scrum board after starting SWD Sprint 1

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Step 3: Create User Stories

Four user stories and two technical tasks were created with acceptance criteria. The stories cover creating requests, validating inputs, saving and closing requests, and reading FAQs. The tasks cover automated tests/build and deployment. Planned responsibilities are recorded below; they are not a claim that each member independently authored commits or reviewed the pull request.

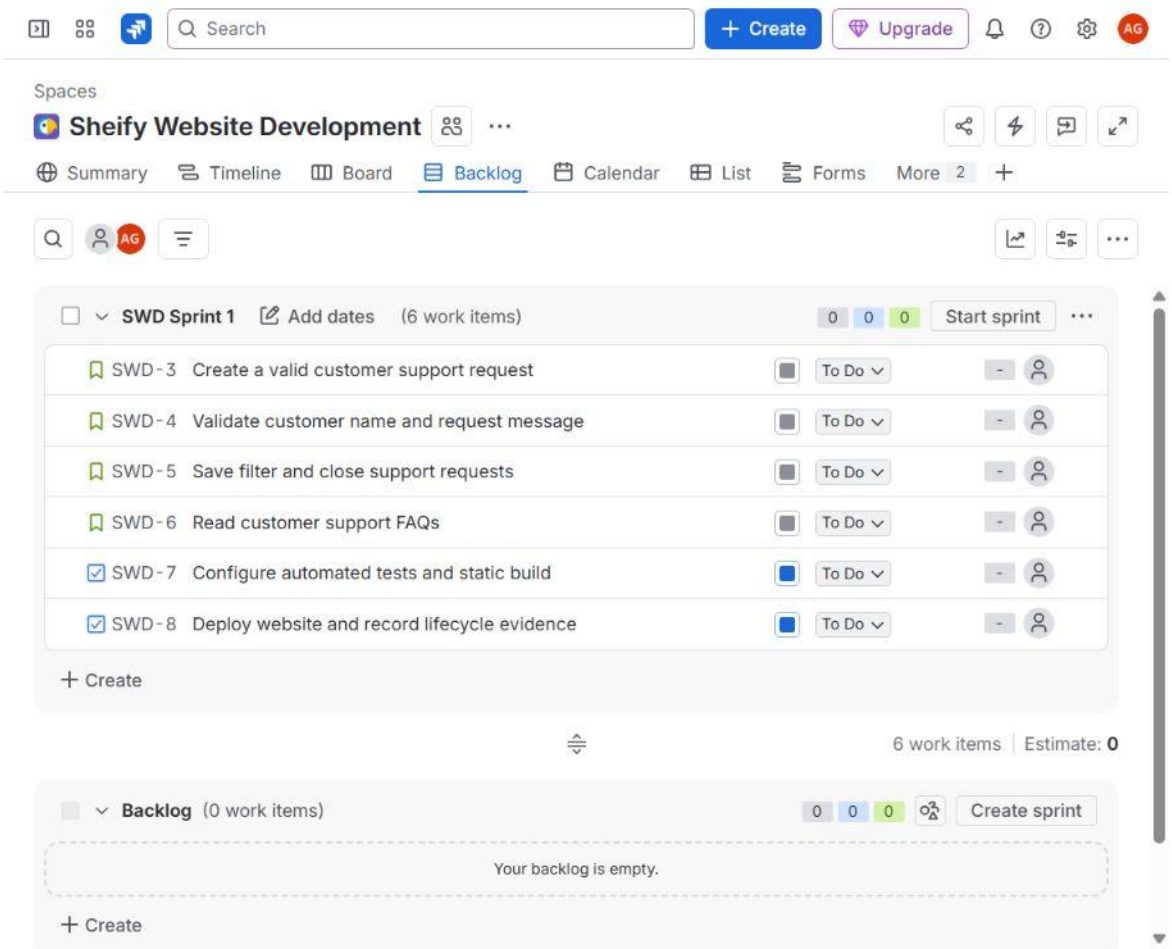


Figure 3: Six work items in the sprint backlog

The sprint contains the following six work items. The Points column records Jira estimates actually entered; a dash means no estimate was entered.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Table 1: Sprint Backlog of Project SWD

Issue Key	Type	Summary	Points	Planned owner
SWD-3	Story	Create a valid customer support request	3	Atharva
SWD-4	Story	Validate name and request message	—	Utsav
SWD-5	Story	Save, filter and close requests	—	Mohit
SWD-6	Story	Read customer support FAQs	—	Utsav
SWD-7	Task	Configure automated tests and build	—	Mohit
SWD-8	Task	Deploy website and record evidence	—	Atharva

Atharva is the recorded Jira assignee of SWD-3. The other work items remain unassigned in Jira, with intended responsibilities recorded in their descriptions. SWD-3 has a three-point estimate. These records should not be interpreted as a fully estimated team velocity.

Step 4: Create and Start the Sprint

SWD Sprint 1 was started on 4 October 2026 with six work items and a goal to demonstrate the support workflow and automated delivery. Jira initially offered a two-week end date of 18 October. The sprint was completed early after this practical demonstration; it is not a two-week delivery measurement.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

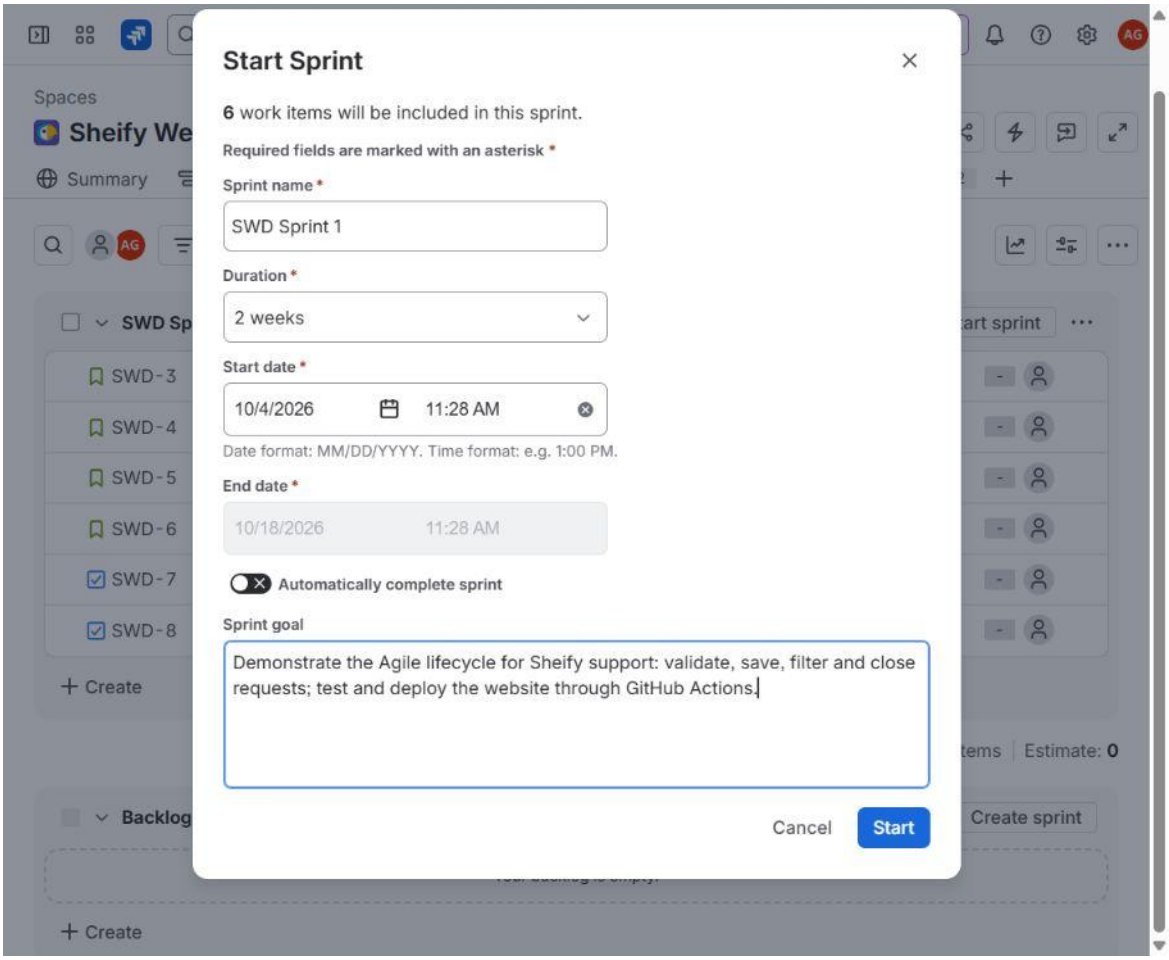


Figure 4: Starting the sprint with its goal and selected work

Step 5: Create a GitHub Account and Repository

The public repository gokhale-atharva/jira-experiment-6-sheify was created on GitHub. It contains the application, ten automated tests, a static build script and the workflow definition. Repository: https://github.com/gokhale-atharva/jira-experiment-6-sheify

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Step 6: Prepare the Development Environment

The application uses Node.js and its built-in test runner. GitHub Actions installs Node.js 22 for a consistent remote environment. The following commands run the project checks from the repository folder:

```
npm test
npm run build
```

Step 7: Create the Application

The application was implemented with HTML, CSS and JavaScript modules. support.mjs contains the request validation and state logic; app.mjs connects that logic to the browser interface. Requests are kept in localStorage in the current browser. The page identifies the group as Atharva, Utsav and Mohit and clearly describes its classroom-demo scope.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

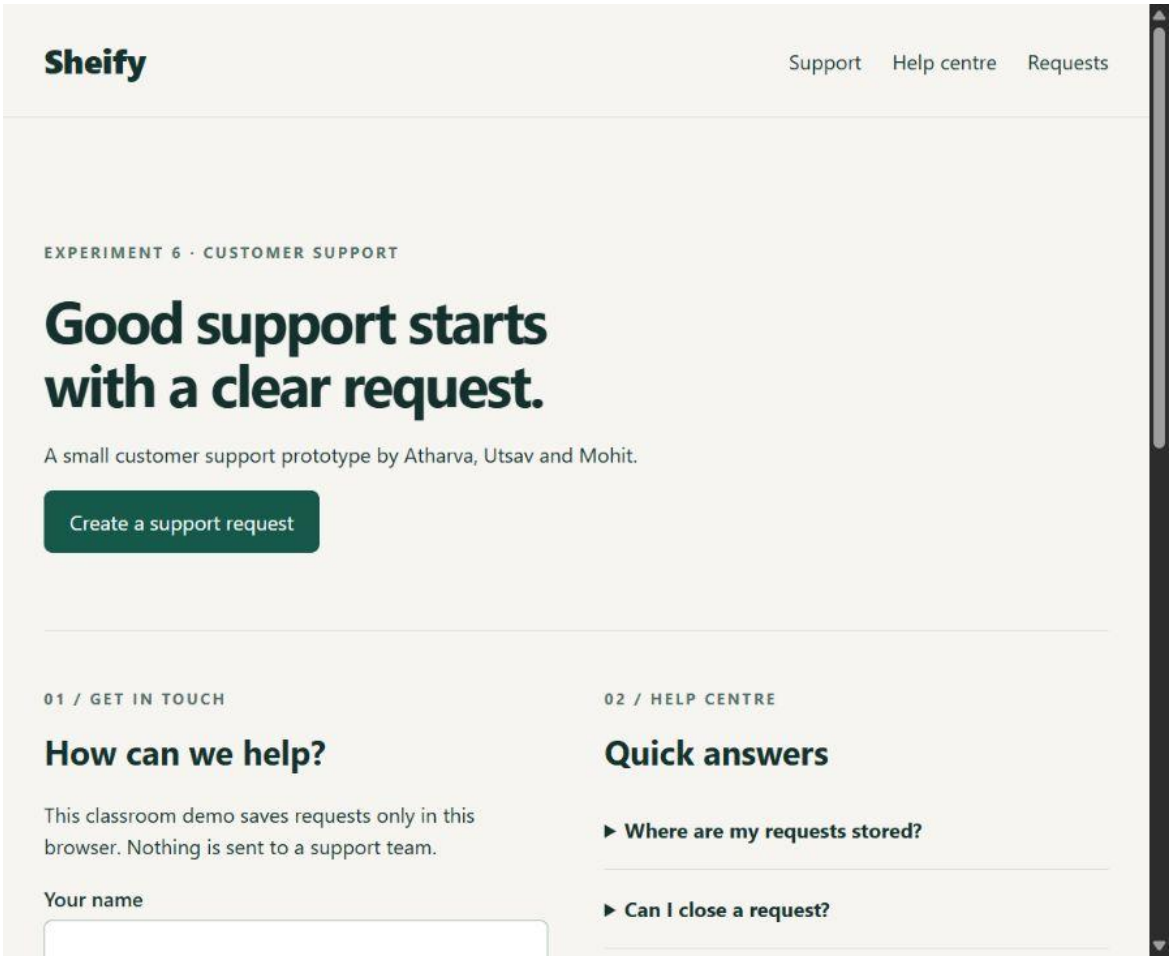


Figure 5: Sheify Customer Support deployed website

The form rejects invalid input. Names must contain 2–80 characters after trimming, and request messages must contain 10–1000 characters. Submitting an empty form displayed the error shown below instead of saving a request.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

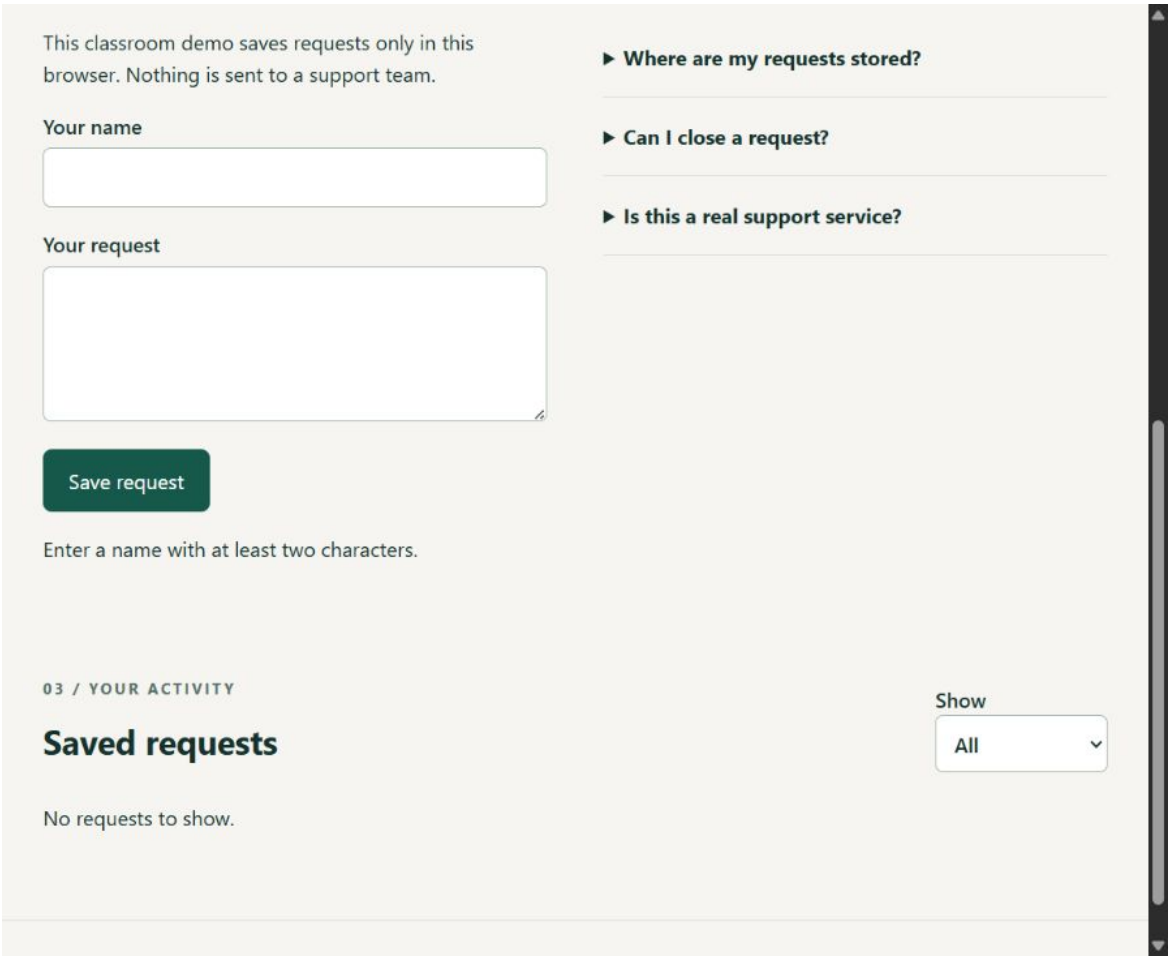


Figure 6: Actual validation error for an empty form

Step 8: Upload the Code to GitHub

The source files were uploaded through GitHub's web interface on the branch SWD-1-customer-support. The source commit and workflow commit include the corresponding Jira epic keys. This execution did not use a local git push.

```
Branch: SWD-1-customer-support
Source commit: 312c24a
```

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

```
SWD-1 Implement customer support prototype and tests
Workflow commit: d8077fe
SWD-2 Add automated tests build and deployment
```

Pull request #1 brought the two commits into main after the automated test-build check passed. The pull request title includes both SWD-1 and SWD-2. No separate teammate approval was recorded on the pull request.

Step 9: Configure GitHub Actions

The file .github/workflows/site.yml was committed to define the pipeline. The workflow uses a hosted Ubuntu runner, checks out the code, installs Node.js 22, runs npm test and builds dist. The deploy job depends on test-build and runs only for a push to main.

```
Workflow: Website CI and deployment
```

Step 10: Connect Source Control and Jira Traceability

The SWD issue keys appear in the branch name, commit messages and pull request #1. This links the implementation history to the Jira epics. The pull request was merged into main as commit 721fb9d. The screenshot shows the actual pull request used for this experiment.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

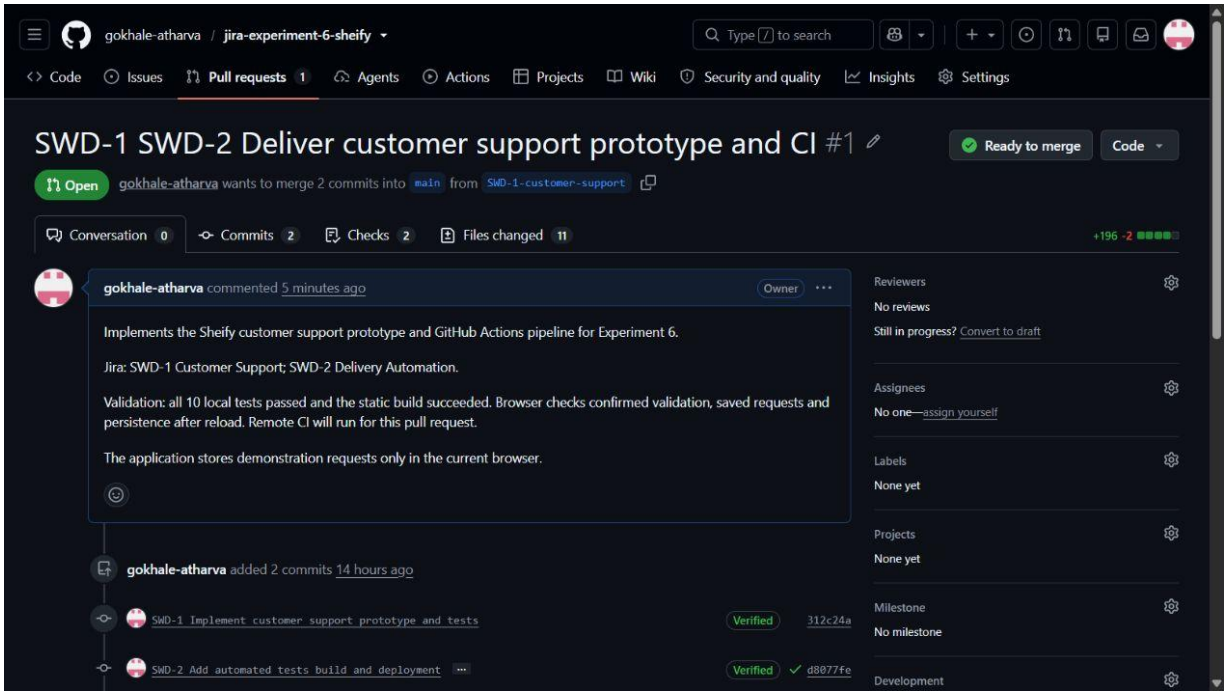


Figure 7: Pull request linking SWD-1 and SWD-2 to the source

Step 11: Test and Build the Project

The pull request triggered the test-build job automatically. All ten tests passed. The tests check valid requests, missing and short fields, maximum and boundary lengths, closing behavior, invalid states and filtering. The static build copies the required application files into dist. The essential workflow configuration is shown below.

```
name: Website CI and deployment
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
permissions:
  contents: read
concurrency:
  group: website-${{ github.ref }}
  cancel-in-progress: true
jobs:
  test-build:
```

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-node@v4
        with:
          node-version: '22'
      - run: npm test
      - run: npm run build
      - uses: actions/upload-pages-artifact@v4
        if: github.event_name == 'push'
        with:
          path: dist
  deploy:
    if: github.event_name == 'push'
    needs: test-build
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pages: write
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/configure-pages@v5
      - id: deployment
        uses: actions/deploy-pages@v4
```

The main-branch run also passed all ten tests with zero failures. Its test-build job succeeded in six seconds. The output below shows the actual remote test summary, including tests 10, pass 10 and fail 0.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

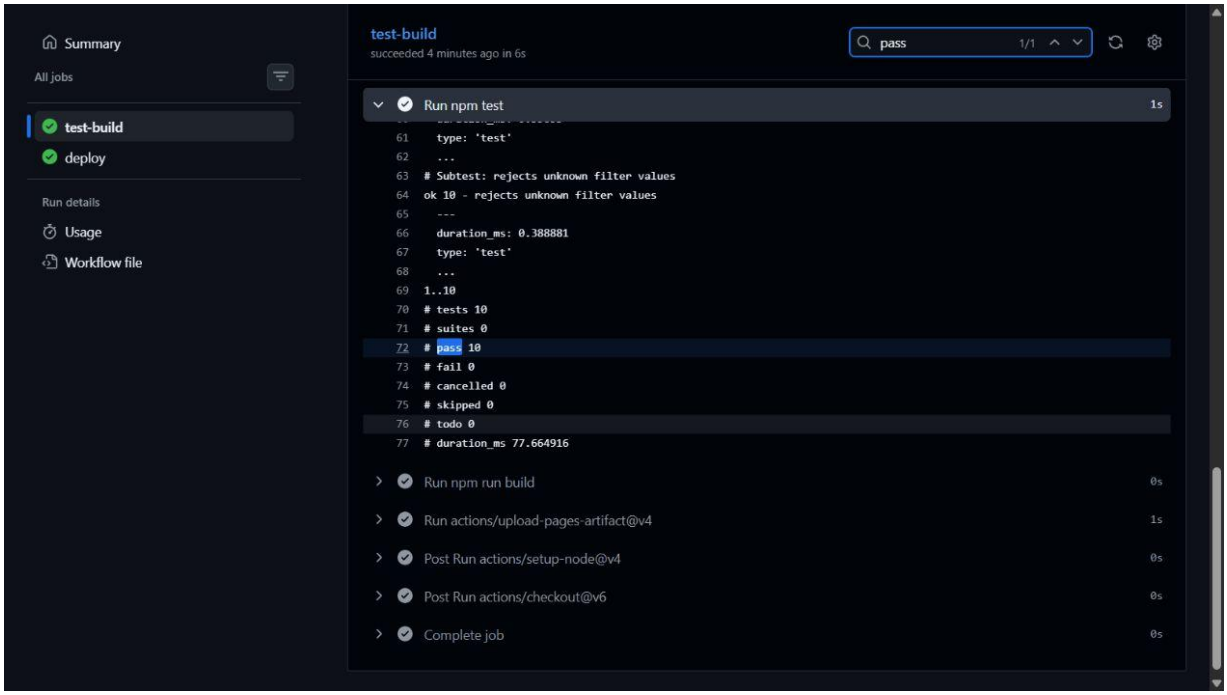


Figure 8: Remote test output with ten passes and zero failures

Step 12: Verify Automatic Build and Deployment

The workflow listens for pull requests targeting main and pushes to main. Run #1 was automatically triggered by opening pull request #1. The deploy job was correctly skipped for that pull request. Run #2 was triggered by the merge commit pushed to main.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

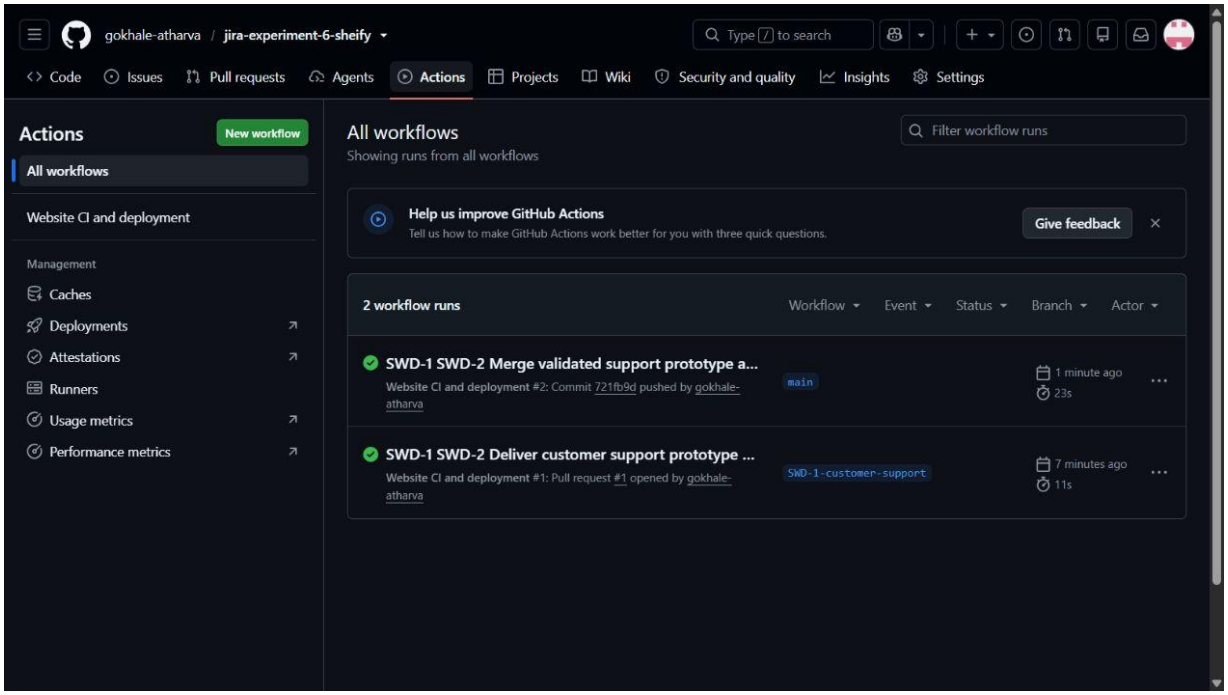


Figure 9: Automatic pull request and push workflow runs

Run #2 completed successfully in 23 seconds. Both test-build and deploy succeeded, and the workflow produced a github-pages artifact. The run summary includes the deployed site URL. GitHub displayed action-runtime deprecation warnings, but the recorded run completed successfully.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

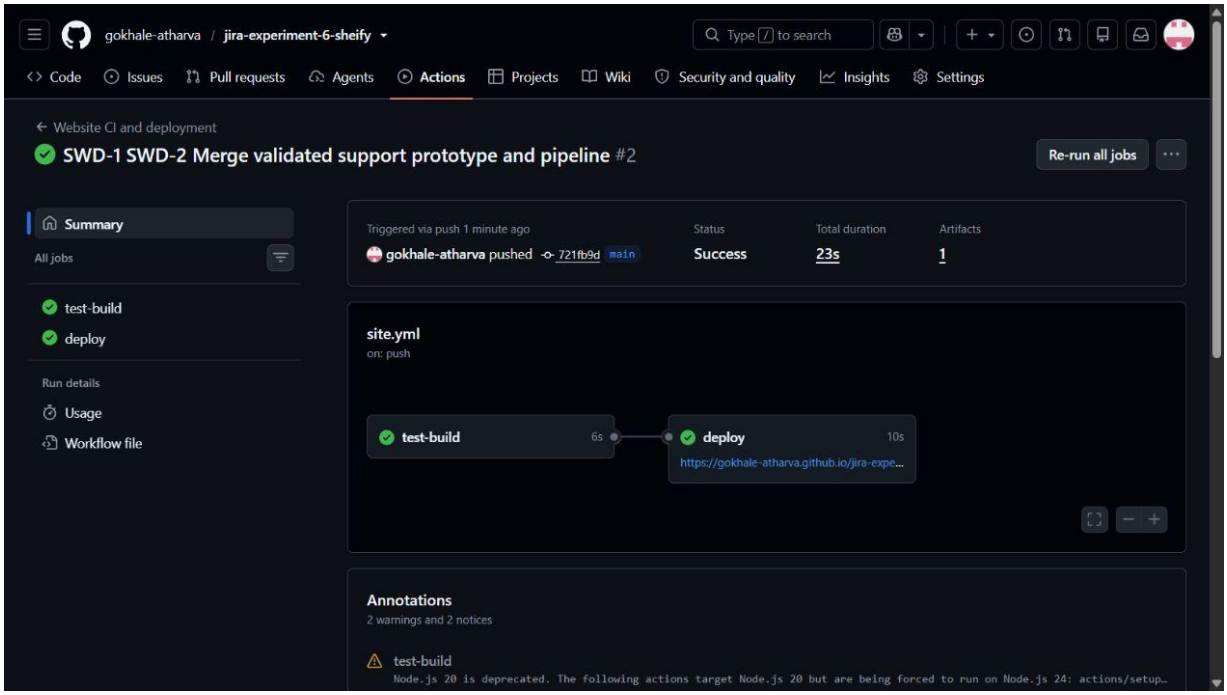


Figure 10: Successful test-build and deploy jobs

Step 13: Update the Jira Workflow

SWD-3 was moved from To Do to In Progress and then In Review to demonstrate the workflow. The screenshot records the review state with Atharva as assignee. After the successful merge, remote checks and deployment, the six completed work items were moved to Done using Jira's bulk status action.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

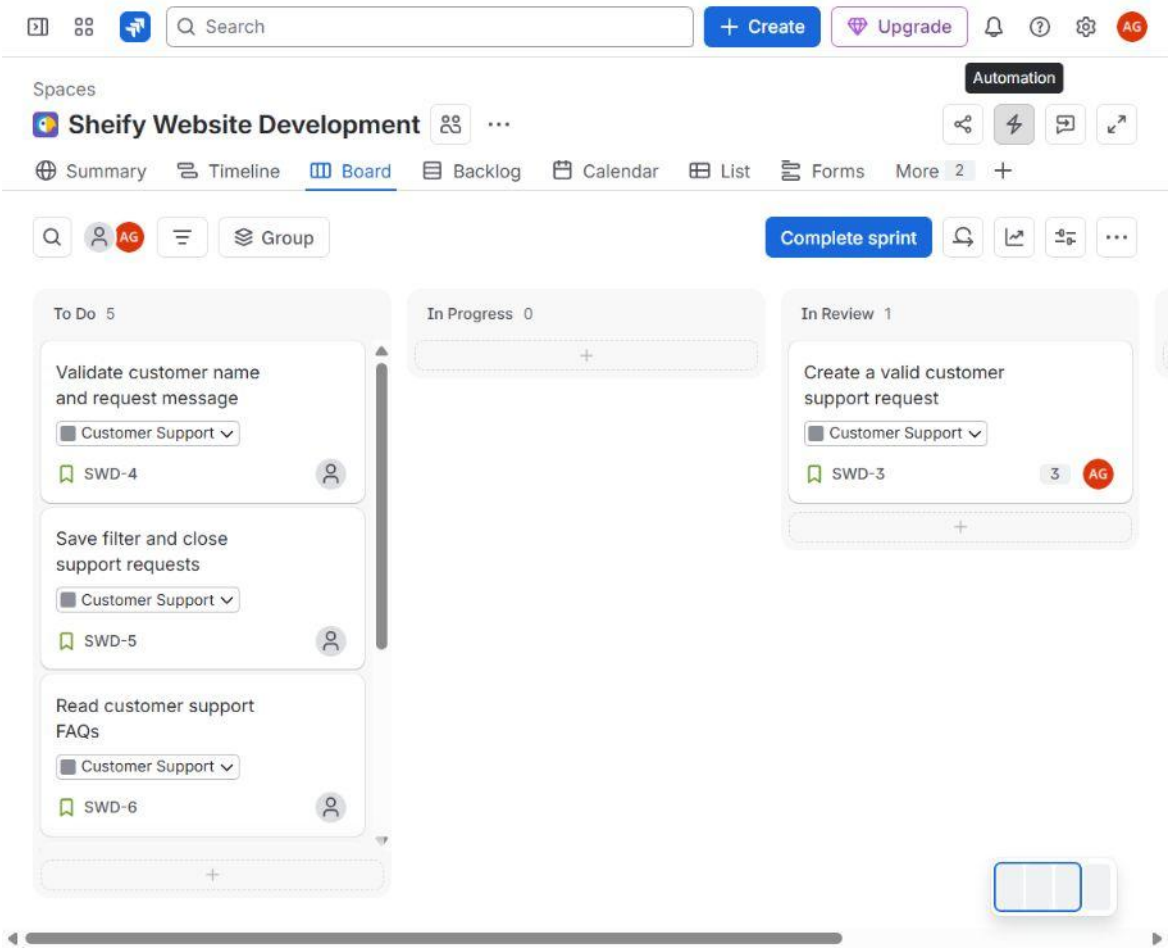


Figure 11: SWD-3 in review with Atharva as assignee

Definition of Done used for this practical:

The implemented behavior satisfies the recorded acceptance criteria, the source is merged into main with Jira-key traceability, automated tests and the static build pass, and the deployed browser application is verified. Jira status was updated manually after these checks. A peer review approval and automatic Jira synchronization are not claimed.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

Step 14: Complete the Sprint

Jira's Complete sprint dialog confirmed six completed work items and no remaining work. The sprint was then completed. The screenshot below records the completion confirmation for SWD Sprint 1.

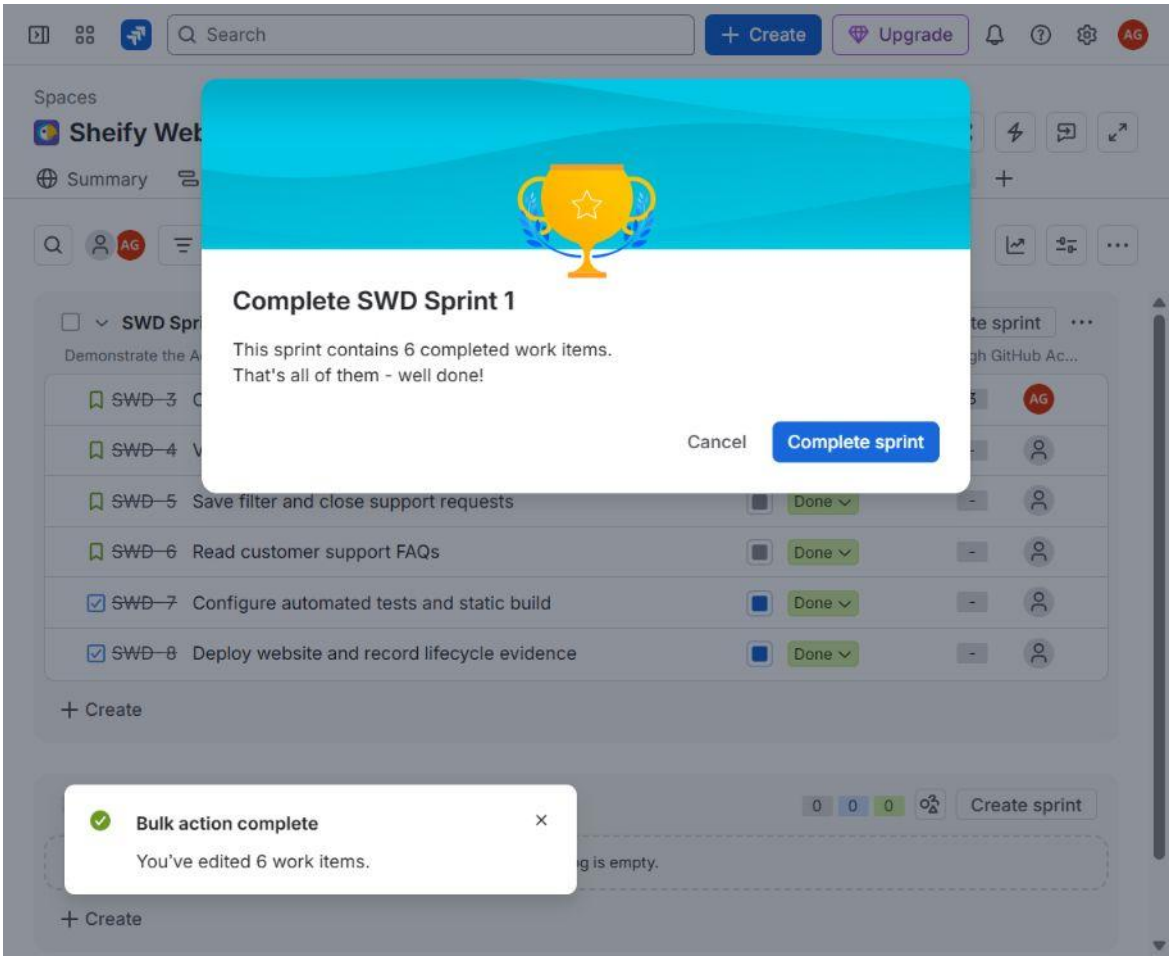


Figure 12: Jira confirms all six sprint work items completed

Sprint result: four stories and two technical tasks completed. Two parent epics organize the work and are not included in the six-item sprint total. Because only one story was estimated, the recorded points do not measure the total effort of the sprint.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

The practical demonstrates planning, implementation, validation and completion within a short classroom exercise. No multi-day burndown or team productivity claim is inferred from this short execution.

Retrospective: define estimates and actual assignees before starting the next sprint; add a separate teammate review; and configure the GitHub for Jira app if automatic development and deployment visibility is required.

Step 15: Verify GitHub Pages Deployment

The repository's Pages source was set to GitHub Actions. The test-build job uploads dist, and the deploy job publishes it with pages: write and id-token: write permissions. The live URL below was opened successfully in a browser.

```
https://gokhale-atharva.github.io/jira-experiment-6-sheify/
```

A request was submitted using the name Atharva and a demonstration message. It remained available after reloading the deployed page, confirming browser persistence. The request was then closed, and selecting the Closed filter displayed the closed record.

```
Observed result: request saved and retained after reload.
Observed result: request closed and shown by the Closed filter.

The application stores demonstration data only in the current browser.
```

The screenshot below shows the final closed request in the deployed application. This completes the practical path from a requirement in Jira to a tested and published increment.

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

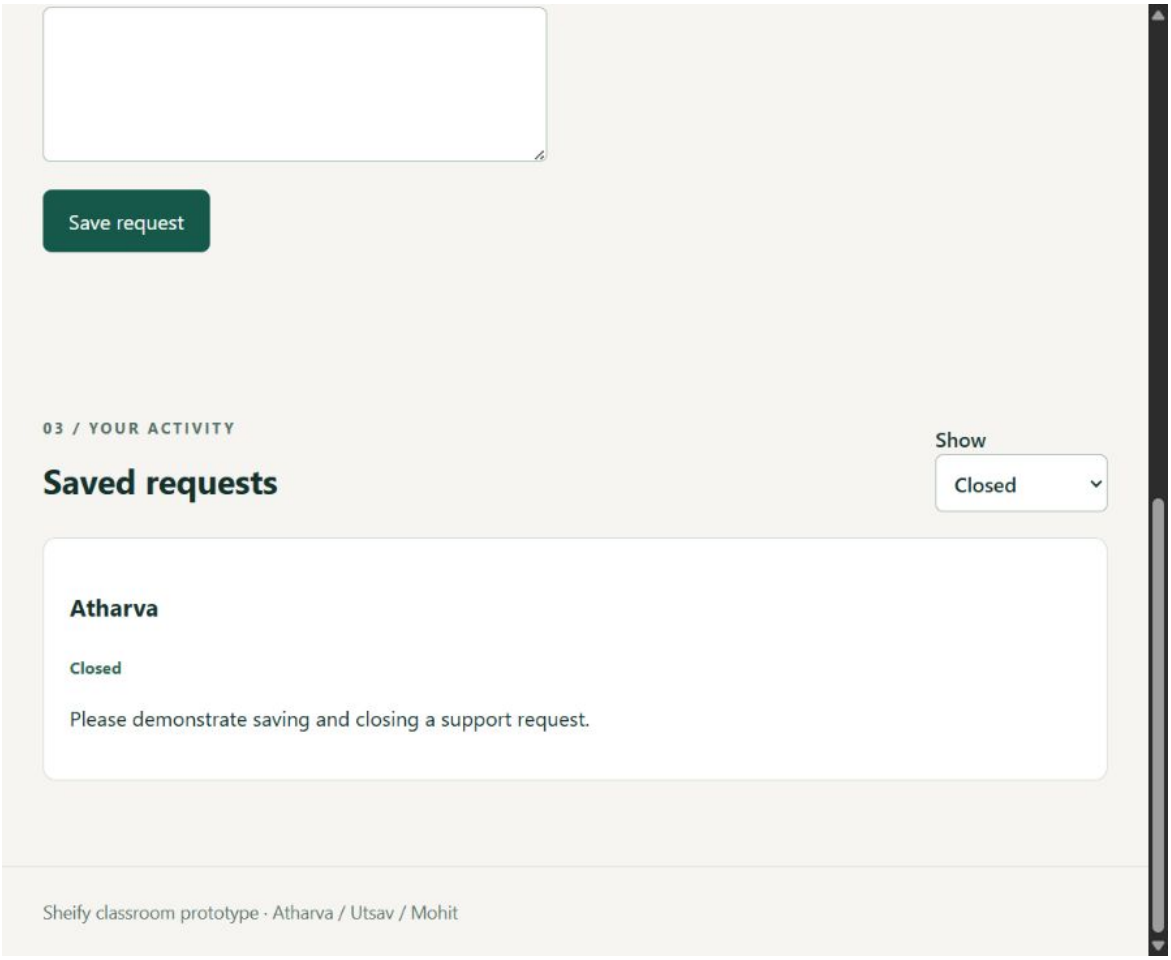


Figure 13: Closed request displayed by the deployed application

CONCLUSION :

The experiment demonstrated an Agile workflow in Jira for the Sheify Customer Support application. The group's project contains two epics, four user stories and two technical tasks. Six work items were selected for SWD Sprint 1, the workflow was demonstrated, and the sprint was completed with all six items done.

Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Atharva Gokhale		**Subject: ASD&D**
T.E. - T1 Batch: T12 Sem: V	**Roll No: 31**	**Date: 04/10/26**

The application and workflow were stored in GitHub with Jira keys in the branch, commits and pull request. GitHub Actions automatically ran the tests and build for the pull request and repeated validation after the merge to main. The main-branch workflow successfully deployed the static artifact to GitHub Pages.

All ten automated tests passed with zero failures. Browser checks demonstrated validation, request creation, persistence after reload, closure and status filtering. The recorded integration uses issue-key traceability and manual Jira transitions; it does not include a configured GitHub for Jira app or a separate teammate review.

The practical provided experience with Scrum project setup, backlog creation, acceptance criteria, sprint tracking, version control, pull requests, automated testing and deployment. The next iteration can improve estimation, member assignment and review practices while retaining the same repeatable delivery pipeline.